

■ END SEM EXAM BLUEPRINT

DBMS (BCSE0402) + DSA-2 (BCS301) · B.Tech 4th Sem

NIET Greater Noida · Units 1–5 · Complete Study Guide

How to use this blueprint:

This guide analyses all 5 units from your lecture slides and ranks every topic by exam importance. ■ **High Priority** topics appear most often in previous papers, weekly assignments, and "Expected University Questions" sections. ■ **Medium Priority** topics are conceptual but less numerically intensive. ■ **Quick Wins** are definitions or one-liners you can master in 15 minutes.

■ 5-Unit Snapshot — What you are dealing with

Unit 1	DSA-2	Trees, BST, AVL, B-Tree, Heap Sort	■■■■
Unit 2	DBMS	Keys, Constraints, SQL Functions, Normalization, FDs	■■■■■
Unit 3	DBMS	Joins, Set Ops, Nested Queries, PL/SQL, Views	■■■■
Unit 4	DBMS	Transactions, ACID, 2PL, Schedules, Deadlock, DCL/TCL	■■■■
Unit 5	DBMS	NoSQL, MongoDB, CAP, BASE vs ACID, Cloud DB	■■■

* Weight based on frequency of questions in previous year papers + instructor assignments

■ UNIT 1 — Trees & Advanced DSA (BCS 301)

This unit is part of DSA-2, not DBMS. However, it carries equal weight. Tree traversals + BST operations + AVL rotations + Heap Sort are all asked numerically almost every year.

■ High Priority

- Tree traversal (Inorder/Preorder/Postorder) with examples
- Construct Binary Tree from two traversals (Inorder+Preorder or Inorder+Postorder)
- BST: Insert, Delete, Search with diagram
- AVL Tree: Insertions & LL/LR/RL/RR rotations
- Heap Sort algorithm + time complexity $O(n \log n)$
- B-Tree: Properties, order, insertion/deletion
- Binary Heap: Min/Max heap operations

■ Quick Wins

- Basic tree terminology (Root, Leaf, Height, Depth, Level)
- Full Binary Tree theorem ($L = I + 1$)
- Complete vs Strictly binary tree definition
- Array & Linked List representation
- Threaded Binary Tree — concept only
- Priority Queue definition

■ Most Expected Questions — Unit 1

1	Define tree data structure with terminologies. (Root, Parent, Child, Sibling, Leaf, Height, Depth)	5–7 M
2	Construct Binary Tree from given Inorder and Preorder / Postorder traversal. Inorder: 10,15,17,18,20,25,30,35,38,40,50 Preorder: 20,15,10,18,17,30,25,40,35,38,50	10 M
3	Create BST using: 28, 56, 12, 9, 87, 7, 23, 19. Perform deletion of 56.	10 M
4	Create AVL Tree using: 39, 56, 2, 9, 87, 7, 23, 19. Show all rotations step by step.	10 M
5	Construct expression tree for $(a + b * c) * (d + e)$. Write inorder, preorder and postorder.	7 M
6	Explain Heap Sort with algorithm. Show sorting steps for $A = [7, 4, 3, 1, 2]$. Time complexity?	10 M
7	What is B-Tree? How is it different from Binary Tree and BST? Properties of B-Tree of order m .	7 M
8	Prove: In non-empty full binary tree, number of leaves = number of internal nodes + 1.	5 M
9	Explain Binary Heap (Min & Max). Write Insert and Extract-Max algorithms.	7 M
10	What is AVL tree? How does it differ from BST? Explain LL, LR, RL, RR rotations with diagrams.	10 M

UNIT 2 — SQL Basics & Normalization (DBMS)

Unit 2 is the highest-weight unit in DBMS. The instructor's slides explicitly list 16+ "Expected Questions for University Examination." Normalization up to BCNF + Functional Dependencies + canonical cover are numerically asked almost every year.

■ High Priority

- All types of Keys with examples (Super, Candidate, Primary, Alternate, Composite, Foreign, Unique)
- Normalization: 1NF → 2NF → 3NF → BCNF with stepwise numerical examples
- Functional Dependencies: definition, types (trivial/non-trivial), attribute closure
- Canonical Cover (Minimal Cover) of FD sets — must know algorithm
- Candidate keys from FD set — numerical
- SQL Aggregate functions: COUNT, SUM, AVG, MAX, MIN with GROUP BY + HAVING
- WHERE vs HAVING clause — very commonly asked distinction
- SQL Constraints: NULL, NOT NULL, DEFAULT, CHECK, UNIQUE, PRIMARY KEY, FOREIGN KEY

■ Quick Wins

- Super key vs Candidate key distinction
- Alternate key definition
- String functions: CONCAT, LENGTH, SUBSTR, TRIM
- Math functions: ABS, ROUND, CEIL, FLOOR
- Advanced functions: RANK, ROW_NUMBER, LEAD, LAG (just concept)
- ORDER BY, DISTINCT, LIMIT, BETWEEN, LIKE predicates
- Difference between RDBMS and File System

■ Most Expected Questions — Unit 2

1	Explain Super Key, Candidate Key, and Primary Key with suitable examples. What is an Alternate Key?	7 M
2	Discuss importance of Foreign Keys in maintaining referential integrity. With diagram.	5 M
3	Given a table with attributes (A, B, C, D) and FDs: $A \rightarrow B$, $B \rightarrow C$, $A \rightarrow D$ — find all candidate keys and check normal forms.	10 M
4	Normalize the following relation to 3NF and BCNF step by step. Explain each normal form.	10 M
5	What is Canonical Cover? Construct canonical cover for FD set: $\{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$. Verify correctness.	10 M
6	Explain the difference between WHERE and HAVING clauses. Write SQL to find departments with average salary > 5000.	7 M

7	Demonstrate use of Aggregate functions (SUM, AVG, COUNT, MAX, MIN) and GROUP BY with examples.	7 M
8	What are constraints in SQL? Explain NULL, NOT NULL, DEFAULT, CHECK with practical examples.	7 M
9	What is Functional Dependency? Explain types (trivial, non-trivial). Find attribute closure of A+ given FDs.	7 M
10	Discuss limitations of 3NF and justify the need for BCNF. What is the difference between 3NF and BCNF?	7 M
11	Design a database schema and normalize to BCNF. Explain impact of normalization on query performance.	10 M

■ UNIT 3 — Complex Queries & PL/SQL (DBMS)

Unit 3 covers advanced SQL and PL/SQL. Joins (all types) are extremely frequently asked with diagram + query. Nested queries (IN, EXISTS) and PL/SQL constructs (Procedure, Trigger, Cursor) are standard 10-mark questions.

■ High Priority

- All JOIN types with SQL syntax + diagram: INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, NATURAL
- Nested Queries / Subqueries: IN, NOT IN, EXISTS, NOT EXISTS, ALL, ANY
- PL/SQL: Procedure vs Function — definition + write a simple one
- Triggers: definition, types (BEFORE/AFTER INSERT/UPDATE/DELETE), syntax, practical example
- Cursors in PL/SQL: definition, types (implicit/explicit), FETCH loop
- Set operators: UNION, UNION ALL, INTERSECT, MINUS with syntax + example
- Views: CREATE VIEW, updatable views, benefits of views

■ Quick Wins

- Cartesian Product (Cross Join) — concept + example
- Theta Join vs Equi Join vs Natural Join distinction
- Synonyms and Sequences in SQL
- Managing Indexes: clustered vs non-clustered
- Database connectivity concept (Java/Python) — likely only 2-3 M
- Division operator in relational algebra

■ Most Expected Questions — Unit 3

1	Explain all types of JOIN operations (INNER, LEFT, RIGHT, FULL OUTER) with SQL syntax and example diagrams.	10 M
2	What is a Nested Query / Subquery? Explain use of IN, NOT IN, EXISTS, NOT EXISTS with SQL examples.	10 M
3	What is a View in SQL? Write SQL to create, modify and drop a view. What are its advantages?	7 M
4	Explain Triggers in PL/SQL — types, syntax, and write a trigger that fires after INSERT on Employee table.	7 M
5	What are Cursors in PL/SQL? Explain implicit vs explicit cursors. Write a PL/SQL block using a cursor.	7 M
6	Explain Stored Procedures and Functions in PL/SQL. Write a procedure to calculate factorial of a number.	7 M
7	Explain UNION, INTERSECT and MINUS operators in SQL with syntax and examples.	7 M

8	What is Natural Join? How does it differ from Equi Join? Explain with example tables.	5 M
9	What is Cartesian Product? Write SQL for cross join. When is it useful?	5 M
10	How does DBMS connectivity work with Java/Python? Explain JDBC steps for connecting to a DB.	5 M

■ UNIT 4 — Transactions & Concurrency Control (DBMS)

Unit 4 is conceptually heavy. ACID properties + Transaction lifecycle + Serializability + 2-Phase Locking are the most frequently asked long-answer questions. Deadlock handling and DCL/TCL commands are asked as short questions.

■ High Priority

- ACID Properties — Atomicity, Consistency, Isolation, Durability with examples (Fund Transfer example)
- Transaction States/Lifecycle diagram: Active→Partially Committed→Committed→Terminated, Failed→Aborted
- Serializability: Conflict Serializability vs View Serializability
- 2-Phase Locking Protocol (2PL): Growing phase, Shrinking phase — ensures serializability
- Strict 2PL and Rigorous 2PL — difference and recoverability guarantee
- Deadlock: detection, prevention, avoidance strategies
- Recoverability: Recoverable vs Cascadeless vs Strict schedules
- Timestamping protocol and Validation-based protocol — concept

■ Quick Wins

- DCL commands: GRANT and REVOKE with SQL syntax
- TCL commands: COMMIT, ROLLBACK, SAVEPOINT with SQL syntax
- Concurrent execution benefits (throughput + response time)
- Schedule types: Serial vs Non-serial
- What are locking techniques — shared lock vs exclusive lock
- PL/SQL DDL vs DML vs DCL vs TCL classification

■ Most Expected Questions — Unit 4

1	How do ACID properties ensure data integrity in a DBMS? Explain each with the bank transfer example.	10 M
2	Draw and explain the complete Transaction State Diagram. Describe each state.	7 M
3	What is Serializability? Distinguish between Conflict Serializability and View Serializability with examples.	10 M
4	Explain the 2-Phase Locking Protocol (2PL). How does it ensure serializability? Difference between 2PL, Strict 2PL and Rigorous 2PL.	10 M
5	What is a Deadlock in DBMS? Explain detection, prevention and avoidance techniques.	7 M
6	Explain Recoverable, Cascadeless and Strict Schedules. Why is recoverability important?	7 M
7	Discuss Timestamping-based Concurrency Control protocol. How does it handle conflicts?	7 M

8	What is DCL? Explain GRANT and REVOKE with SQL syntax and examples.	5 M
9	Explain COMMIT, ROLLBACK and SAVEPOINT in SQL with examples. What is TCL?	5 M
10	What are Locking Techniques in concurrency control? Explain shared lock, exclusive lock and lock compatibility matrix.	7 M
11	Define DDL, DML, DCL, TCL commands with examples. How do they differ from each other?	5 M

UNIT 5 — NoSQL & Cloud Database (DBMS)

Unit 5 is the easiest to score in. Most questions are conceptual/definition-based. Learn NoSQL types + CAP theorem + ACID vs BASE + MongoDB basics. Very few numerical problems.

■ High Priority

- NoSQL vs RDBMS: key differences table (Schema, Scalability, Transactions, Data Model)
- CAP Theorem: Consistency, Availability, Partition Tolerance — and trade-offs
- BASE vs ACID: what is BASE? (Basically Available, Soft state, Eventually consistent)
- Types of NoSQL Databases: Key-Value, Document, Column, Graph — with examples
- MongoDB: Introduction, features, BSON format, CRUD operations (insertOne, find, updateOne, deleteOne)
- MongoDB vs RDBMS concepts: Collection=Table, Document=Row, Field=Column

■ Quick Wins

- MongoDB Atlas, Stitch, Cloud Manager — one-liner each
- Horizontal vs Vertical Scaling — with NoSQL context
- Polyglot Persistence — definition
- Cassandra, HBase, Neo4j, Riak — just know which type each is
- MongoDB Shell commands — basic syntax
- MongoDB operators: \$gt, \$lt, \$eq, \$and, \$or — quick reference

■ Most Expected Questions — Unit 5

1	What are NoSQL databases? What are the different types? Explain each with example databases.	10 M
2	Explain the CAP Theorem. Why can't a distributed system satisfy all three properties simultaneously?	7 M
3	Compare ACID and BASE properties. In what scenarios would you prefer NoSQL over RDBMS?	7 M
4	Compare NoSQL databases with RDBMS (table format). When should you use NoSQL?	7 M
5	What is MongoDB? Explain its features. How is it different from relational databases?	7 M
6	Explain CRUD operations in MongoDB with syntax examples: insertOne, find, updateOne, deleteOne.	7 M
7	What is horizontal vs vertical scaling? How does MongoDB achieve horizontal scaling using sharding?	5 M
8	What are the advantages of BSON over JSON in MongoDB?	5 M
9	Explain MongoDB Atlas. What are its key features for cloud database management?	5 M
10	What is Polyglot Persistence? When and why is it used?	3 M

■ Your 10-Day Master Preparation Schedule

Rule: Spend the most time where marks are guaranteed. Unit 2 (Normalization) and Unit 1 (Tree numericals) reward effort directly. Unit 5 is easy marks — cover it last quickly.

Day 1	Unit 1 — Basics + Traversals	Master tree terminology. Practice 5 traversal problems. Construct trees from Inorder+Preorder.	4–5 hrs
Day 2	Unit 1 — BST + AVL	Insert & delete in BST (with diagrams). All 4 AVL rotations (LL/LR/RL/RR) with practice problems. Build AVL from given set.	4–5 hrs
Day 3	Unit 1 — Heap Sort + B-Tree	Heap Sort algorithm: BUILD_HEAP + HEAPIFY. Trace through 5 examples. B-Tree properties and insertion.	4 hrs
Day 4	Unit 2 — Keys + Constraints + SQL	Learn all key types with diagrams. SQL constraints. Write GROUP BY + HAVING queries. WHERE vs HAVING.	4–5 hrs
Day 5	Unit 2 — FDs + Normalization	Functional Dependencies: closure, canonical cover. 1NF→2NF→3NF→BCNF with 3 full numerical examples.	5–6 hrs
Day 6	Unit 2 Revision + Unit 3 — Joins	Revise normalization numericals. Then: all JOIN types with SQL queries and result tables.	5 hrs
Day 7	Unit 3 — PL/SQL + Views	Nested queries (IN/EXISTS). Write Procedure, Function, Trigger, Cursor in PL/SQL. CREATE VIEW examples.	4–5 hrs
Day 8	Unit 4 — Transactions + ACID	ACID with bank example. Transaction lifecycle diagram. Schedule types. Serializability (conflict + view).	4–5 hrs
Day 9	Unit 4 — Concurrency + Unit 5	2PL protocol, deadlock techniques. DCL/TCL commands. Then: all of Unit 5 (NoSQL, CAP, BASE, MongoDB CRUD).	4–5 hrs
Day 10	Full Revision + Previous Papers	Attempt 2 full previous year papers. Mark weak areas. Revise formulae, algorithms, definitions.	5–6 hrs

■ Formulae, Properties & Quick-Reference Card

Max nodes at level l	2^l
Max nodes in binary tree of height h	$2^{h+1} - 1$
Min height for N nodes	$\lceil \log_2(N+1) \rceil - 1$
Full BT: Leaves vs Internal	$L = I + 1$; $N = 2I + 1$; $N = 2L - 1$

Heap Sort time complexity	$O(n \log n)$ — worst, best and average
AVL: Balance Factor	$BF = \text{Height}(\text{Left}) - \text{Height}(\text{Right})$. Valid: -1, 0, +1
B-Tree of order m	Each non-root node: $\lceil m/2 \rceil - 1$ to $m - 1$ keys; root: 1 to $m - 1$ keys
Canonical Cover	Step 1: Decompose RHS $\rightarrow$ single attr. Step 2: Remove extraneous attrs. Step 3: Remove redundant FDs
1NF	All attributes are atomic (no repeating groups)
2NF	1NF + No partial dependency (every non-key attr fully depends on PK)
3NF	2NF + No transitive dependency
BCNF	For every FD $X \rightarrow Y$: X must be a superkey
ACID	Atomicity · Consistency · Isolation · Durability
BASE	Basically Available · Soft state · Eventually consistent
CAP Theorem	Consistency · Availability · Partition Tolerance — pick only 2
2PL Phases	Growing phase: only LOCK. Shrinking phase: only UNLOCK. Lock point = end of growing phase.
Serial Schedule	Transactions execute one after another — always serializable
Conflict Serializable	Schedule equivalent to serial schedule by swapping non-conflicting operations

■ Scoring Strategy — How to Maximise Your Marks

Do ALL numericals from Unit 1 & 2 first. They are worth 10 marks each and most students skip them. BST + AVL + Tree Construction + Normalization together can give you 40+ marks.

Learn to draw diagrams. Examiners award marks for clear diagrams — Transaction State Diagram, AVL rotation diagrams, ER $\rightarrow$ Relational mapping. Label every node/state.

Unit 5 is the "buffer unit". NoSQL questions are mostly 5-7 marks and conceptual. Master the CAP theorem + ACID vs BASE table + MongoDB CRUD and you will score full marks here easily.

Unit 4 (Transactions): The bank transfer example (transfer \$50 from A to B) appears in almost every answer for ACID. Memorise it and use it to explain Atomicity, Consistency, Isolation and Durability.

For 10-mark questions: Always give Definition $\rightarrow$ Properties/Types $\rightarrow$ Example $\rightarrow$ Diagram $\rightarrow$ Conclusion. This structure alone can earn you 8/10 even with partial knowledge.

Previous year paper pattern: Based on the slides, the exam usually has 2 sections — (A) Short answers $2 \times 10 = 20$ marks, (B) Long answers $5 \times 10 = 50$ marks (choose 5 from 8). Normalisation, Joins, ACID, AVL, and NoSQL appear in Section B almost every year.

■ Best of luck! You've got this. Consistent practice of numericals is the key to top scores.